# Ethereum

Covered In CT-03 Merged Slide

# Etherium

**Ethereum** Blockchain Technology is a decentralized, open-source blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (dApps)**.
It allows anyone to write and deploy code (called smart contracts) that runs exactly as programmed, without any possibility of fraud, censorship, or third-party interference.
It was proposed in **2013** by **Vitalik Buterin** and launched in **2015**.

# Ethereum Features

**Smart Contracts** – Self-executing agreements with predefined rules written in code.

Example: A crowdfunding contract automatically refunds contributors if the goal isn't met.

**Ether (ETH)** – The native cryptocurrency used to pay for transactions and computational services on the network.

**Decentralized Applications (dApps)** – Applications that run on the Ethereum network instead of a single centralized server.

**Ethereum Virtual Machine (EVM)** – The runtime environment that executes smart contracts.

**Consensus Mechanism** – Ethereum uses Proof of Stake (PoS) (after the "Merge" update) to validate transactions and secure the network.

**Public Ledger** – All transactions are recorded in a transparent and immutable blockchain.

# Challenges in Implementing Smart Contracts

## Nontechnical Parties

Smart contracts are written in programming languages (e.g., Solidity). Nontechnical people (business owners, lawyers, customers) often cannot understand or verify the code. This creates communication gaps and dependence on technical experts.

**Example:**

A landlord and a tenant agree on rent payments through a smart contract. The landlord is not a programmer and cannot read Solidity code. If the developer encodes a wrong term—e.g., rent due every *15 days* instead of *30 days*—the landlord may not notice until the contract executes automatically.

# Challenges in Implementing Smart Contracts

## Off-Chain Resources

Smart contracts operate on-chain, but they often need real-world data such as prices, weather conditions, or shipment status. Since blockchains cannot fetch external data directly, they rely on **oracles**, which may introduce errors or trust issues.

**Example:**

A crop insurance contract pays farmers automatically if rainfall is below a certain level. The contract relies on a weather oracle. If the oracle reports incorrect rainfall data due to a technical glitch, farmers may either get **unjust payments** or **be denied legitimate claims**.

# Challenges in Implementing Smart Contracts

## Final Agreement Uncertainty (Text vs. Code Conflict)

Traditional contracts use written text. Smart contracts use code. If the written agreement and the code encode different rules, it becomes unclear which one has legal authority.

**Example:**

A written agreement says payment will be released after **10 days**, but the smart contract code releases it after **7 days**. If the buyer claims the release was premature, disputes arise:

Should the court follow the written text or the executed code?

# Challenges in Implementing Smart Contracts

## Automation Conflicts with Real Business Practices

Smart contracts execute automatically and strictly. Real business situations often allow flexibility, negotiation, or forgiveness.

**Example:**

A supplier is late by 1 hour due to a transportation issue. A normal contract might allow leniency. But a **smart contract** may automatically apply a penalty or cancel the agreement, even though both parties would have preferred flexibility.

# Challenges in Implementing Smart Contracts

## Modifying and Terminating Smart Contracts

Blockchains are immutable. Once deployed, smart contracts are very difficult or expensive to change or stop.

**Example:**

Two companies sign a smart contract for long-term service. Later they want to modify the service fee. However, the contract is immutable—they must either: deploy a new contract, or use a complex upgrade pattern. Both options are costly and technically challenging.

# Challenges in Implementing Smart Contracts

## Objectivity vs. Ambiguity

Smart contracts demand precise, objective conditions. Traditional contracts deliberately use flexible or subjective terms like “reasonable time” or “best effort.”

**Example:**

A service contract says the provider must respond in a “reasonable time.” Smart contracts cannot encode “reasonable”—they need a fixed number, like **24 hours**. This removes the natural flexibility of human agreements.

# Challenges in Implementing Smart Contracts

## Payment Guarantees

Smart contracts execute payments only if the digital wallet has enough funds. If the user forgets to deposit funds in the wallet, the contract fails.

**Example:**

A smart contract is supposed to pay freelancers automatically when work is completed. But the employer's wallet has insufficient funds on the payment date.

Result: The contract fails, and payment is not transferred—causing delays and disputes.

# Challenges in Implementing Smart Contracts

## Allocation of Risk

Smart contracts introduce new types of risks:

coding mistakes, vulnerabilities, hacks, oracle failures, gas price spikes

Parties must decide who bears these risks.

**Example:**

A smart contract has a minor coding bug that locks funds permanently.

Who takes responsibility?

the business?

the smart contract developer?

both parties?

Without clear terms, conflicts arise.

# Challenges in Implementing Smart Contracts

## Governing Law and Venue

Smart contracts operate across global blockchains, but legal systems are territorial. Parties must specify:

- ❖ which country's laws apply
- ❖ where disputes will be settled

**Example:**

A Bangladeshi buyer and a German seller use a smart contract. Dispute arises:

- ❖ Which country’s law applies?
- ❖ Where should the case be filed—Dhaka or Berlin?
- ❖ Without an agreed clause, international disputes become complicated.

# Challenges in Implementing Smart Contracts

## Best Practices (Hybrid Approach & Legal Clarity)

Smart contracts are still evolving. Reliable standards and legal frameworks are not yet fully mature. Best practice today is a **hybrid model**:

- The legal text describes the agreement.
- The smart contract executes certain parts.

**Example:**

A commercial lease contract might use:

- Text contract for obligations, dispute resolution, governing law
- Smart contract for automated rent payments

This combination keeps automation while preserving legal clarity.

# Types of Blockchain oracles

There are various types of blockchain oracles, ranging from simple software oracles to complex hard ware-assisted and decentralized oracles.

We can categorize oracles into two categories:

- ❑ Inbound oracles
- ❑ Outbound oracles.

## Inbound oracles

This class represents oracles that receive incoming data from external services and feed it into the smart contract.

- ❖ Software oracles
- ❖ Hardware oracles
- ❖ Computation oracles
- ❖ Aggregation-based oracles
- ❖ Crowd wisdom-driven oracles
- ❖ Decentralized oracles
- ❖ Smart oracles

# Types of Blockchain oracles

## Software oracles

These oracles are responsible for acquiring information from online services on the internet. This type of oracle is usually used to source data such as weather information, financial data (stock prices, for example), travel information, and other types of data from third-party providers.

The data source can also be an internal enterprise system, which may provide some enterprise-specific data. These types of oracles can also be called standard or simple oracles.

A problem with these oracles is that as they are based simply on an online source of data, such as a website; if the source is not live anymore or has failed for some reason, then the feed to the blockchain smart contract will fail.

# Types of Blockchain oracles

## Illustrative Example

### Scenario

Imagine a **crop insurance smart contract** for farmers.

### How it works

The contract checks rainfall data.

If rainfall is **less than 10mm for 10 days**, it automatically **pays compensation** to the farmer.

### Where the data comes from

The smart contract does **not** know rainfall data by itself. It uses a software oracle that pulls data from a website.

# Types of Blockchain oracles

## What goes wrong

Now suppose:

the weather website **shuts down**, or

the server **crashes**, or

the API **stops responding**

**Then:**

The oracle **cannot send rainfall data** to the blockchain

The smart contract **cannot check its condition,** Even if rainfall was actually low, **no compensation is triggered,** So the system fails **not because of the blockchain**, but because the **data source failed**.

# Types of Blockchain oracles

## Hardware oracles

This type of oracle is used to source data from hardware sources such as IoT devices or sensors. This is useful in use cases such as insurance-related smart contracts where telemetry sensors provide certain information, for example, vehicle speed and location. This information can be fed into the smart contract dealing with insurance claims and payouts to decide whether to accept a claim or not. Based on the information received from the source hardware sensors, the smart contract can decide whether to accept or reject the claim.

These oracles are useful in any situation where real-world data from physical devices is required. However, this approach requires a mechanism in which hardware devices are tamper-proof or tam per-resistant. This level of security can be achieved by providing cryptographic evidence (non-repudiation and integrity) of an IoT device's data and an anti-tampering mechanism on the IoT device, which renders it useless if there are any tampering attempts.

# Types of Blockchain oracles

## Computation oracles

These oracles allow computing-intensive calculations to be performed off-chain. As blockchain is not suitable for performing compute-intensive operations, a blockchain (that is, a smart contract on a blockchain) can request computations to be performed on off-chain, high-performance computing infrastructure and get the verified results back via an oracle.

Some blockchain tasks require **heavy computation** (e.g., large mathematical calculations, simulations, video processing). But blockchains like Ethereum are **not designed** for this because:

- computation is **slow and expensive**
- every node would need to repeat the same calculation
- gas costs would be extremely high

So instead, the smart contract can **send the task off-chain** to a powerful computing system. An **oracle** then returns the result **with proof** that it is correct.

**Truebit** is an example of such an oracle-based system.

# Types of Blockchain oracles

## Aggregation-based oracles

In this scenario, a single value is sourced from many different feeds. As an example, this single value can be the price of a financial instrument, and it can be risky to rely upon only one feed. To mitigate this problem, multiple data providers can be used where all of these feeds are inspected, and finally, the price value that is reported by most of the feeds can be picked up. The assumption here is that if the majority of the sources report the same price value, then it is likely to be correct. The collation mechanism depends on the use case: sometimes it's merely an average of multiple values, sometimes a median is taken of all the values, sometimes it's a mean value, and sometimes it's the maximum value.

Care must be taken here because a single malicious node can skew the results of a mean value by reporting a very large or small number. Usually, median values are used in practice because if most nodes are honest, the median value can be reasonably trusted.

# Types of Blockchain oracles

## Crowd wisdom-driven oracles

This is another way that the blockchain oracle problem can be addressed where a single source is not trusted. Instead, multiple public sources are used to eventually deduce the most appropriate data. It solves the problem where a single source of data may not be as trustworthy or accurate as expected. If there is only one source of data, it can be unreliable and risky to rely on entirely. It may turn malicious or become genuinely faulty. In this case, to ensure the credibility of data provided by third-party sources for oracles, data is sourced from multiple sources. These sources can be users of the system or even members of the general public who have access to and have knowledge of some data, for example, a political event or a sporting event where members of the public know the results and can provide the required data. Similarly, this data can be sourced from multiple different news websites.

# Types of Blockchain oracles

## Decentralized oracles

Another type of oracle, which primarily emerged due to decentralization requirements, is a decentralized oracle. As blockchain platforms such as Bitcoin and Ethereum are fully decentralized, it is expected that oracle services should also be decentralized.

This type of oracle can be built based on a distributed mechanism. It can also be envisaged that the oracles can find themselves source data from another blockchain, which is driven by a distributed consensus, thus ensuring the authenticity of data. For example, one institution running its private blockchain can publish its data feed via an oracle that can then be consumed by other blockchains.

# Types of Blockchain oracles

## Outbound oracles

This type, also called a reverse oracle, is used to send data out from the blockchain smart contracts to the outside world. There are two possible scenarios here; one is where the source blockchain is a producer of some data such as blockchain metrics, which are needed for some other blockchain. The actual data somehow needs to be sent out to another blockchain smart contract. The other scenario is that an external hardware device needs to perform some physical activity in response to a transaction on-chain.